



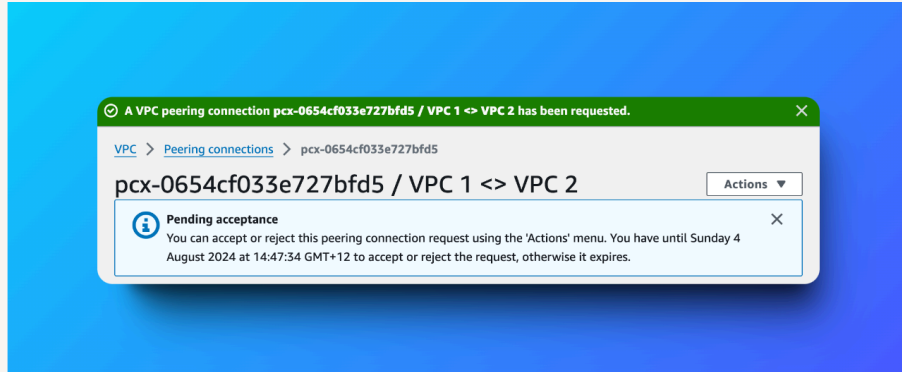
nextwork.org

VPC Peering



Muhammad Sadique

<https://www.linkedin.com/in/muhammad-sadique->





Introducing Today's Project!

What is Amazon VPC?

Amazon VPC (Virtual Private Cloud) is a service that allows you to create a logically isolated section of the AWS Cloud where you can launch AWS resources, like Amazon EC2 instances, in a virtual network you define. It's essentially a virtual datacenter within AWS, giving you control over your network environment. You can customize IP address ranges, subnets, route tables, and security groups.

How I used Amazon VPC in this project

I used VPC for 'VPC Peering' in my today's project.

One thing I didn't expect in this project was...

I was not expecting explicit route table modification required for VPC peering.

This project took me...

90 minuts.



In the first part of my project...

Step 1 - Set up my VPC

In this step we are using VPC resource map to create our VPC resources really fast.

Step 2 - Create a Peering Connection

In this step we are creating peering connection between both VPCs.

Step 3 - Update Route Tables

In this step we are going to modify our route tables to set up a way for traffic coming from VPC 1 to get to VPC 2 and vice-versa.

Step 4 - Launch EC2 Instances

We are going to launch EC2 instances in each of the VPC public subnets.



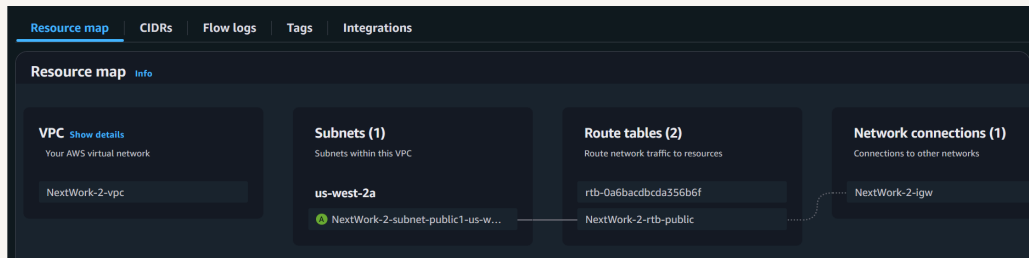
Multi-VPC Architecture

I started my project by launching VPC creation wizard and chose 'VPC & More'. I created 1 public subnet in that VPC.

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 & 10.2.0.0/16. They have to be unique because VPC peering requires that CIDR blocks of both VPCs should not overlap.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances as with EC2 Instance Connect, AWS actually manages a key pair for us! We don't need to manage key pairs ourselves.



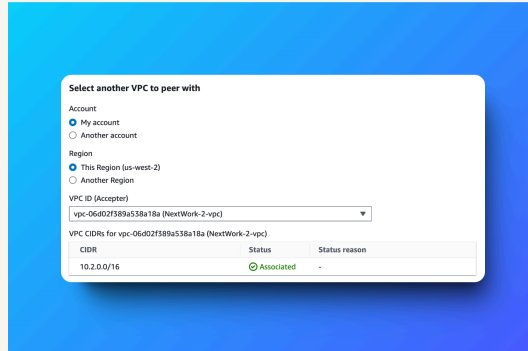


VPC Peering

A VPC peering connection is a networking connection that enables instances in two different Virtual Private Clouds (VPCs) to communicate with each other using their private IP addresses, as if they were on the same network. This allows for secure and private communication between resources in different VPCs, regardless of whether they belong to the same AWS account, project, or region.

Peering connections exist to facilitate direct, cost-effective, and efficient data exchange between different networks, whether they are within a single cloud provider (like AWS VPC peering) or span multiple networks (like internet peering). This direct connection bypasses the need to route traffic through third-party networks, reducing latency and costs associated with transit fees.

In VPC peering, the requester is the VPC that initiates the peering request, while the acceptor is the VPC that receives and accepts (or rejects) the request. Essentially, the requester starts the connection process, and the acceptor determines whether to allow it.

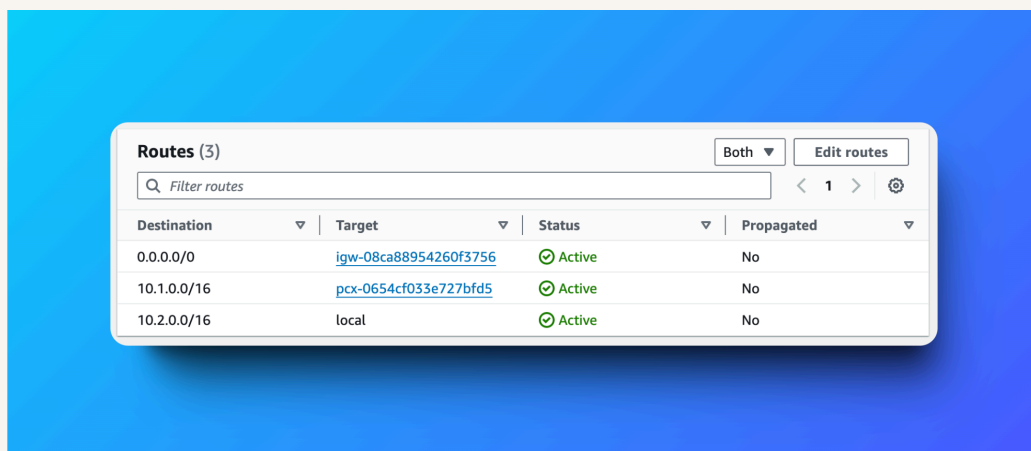




Updating route tables

During VPC Peering, a new route is added to the route tables of both VPCs to enable traffic to flow between them. This route specifies that traffic destined for the CIDR block of the peered VPC should be routed through the VPC peering connection. Essentially, you're telling each VPC how to reach the other via the peering connection.

My VPCs' new routes have a destination of 10.2.0.0/16 in VPC-1 route table and 10.1.0.0/16 in VPC-2 route table. The route target was 'Peering Connection' that i created earlier.



Destination	Target	Status	Propagated
0.0.0.0/0	igw-08ca88954260f3756	Active	No
10.1.0.0/16	pcx-0654cf033e727bfd5	Active	No
10.2.0.0/16	local	Active	No



In the second part of my project...

Step 5 - Use EC2 Instance Connect

In this step we are going to use EC2 Instance Connect to connect to our first EC2 instance.

Step 6 - Connect to EC2 Instance 1

We are try to reconnect to our VPC-1 EC2 instance using EC2 Instance connect from AWS Management Console.

Step 7 - Test VPC Peering

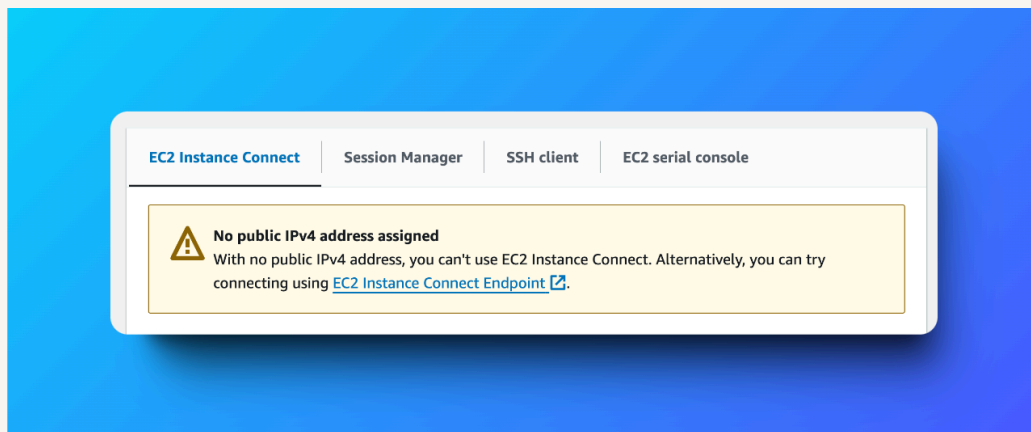
In this step we will try to connect to EC2 Instance in VPC-2 through EC2 Instance in VPC-1 to see if the VPC peering is working properly.



Troubleshooting Instance Connect

Next, I used EC2 Instance Connect to log into my EC2 instance through Public IP that I just assigned.

I was stopped from using EC2 Instance Connect as there was no public IP assigned to it.

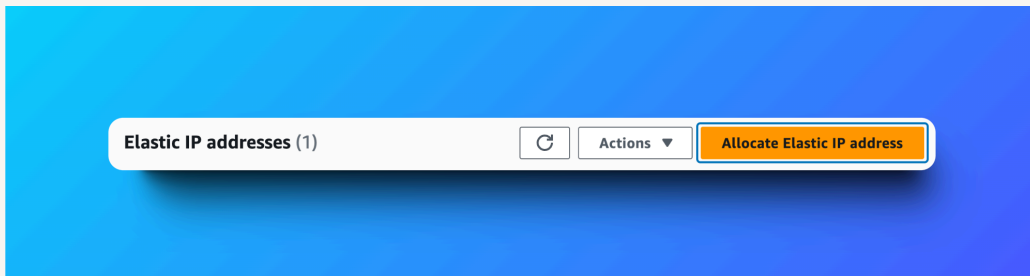




Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are static public IP address that could be assigned to EC2 Instance.

Associating an Elastic IP address resolved the error because now EC2 Instance Connect could reach my EC2 Instance through public IP that i just assigned.

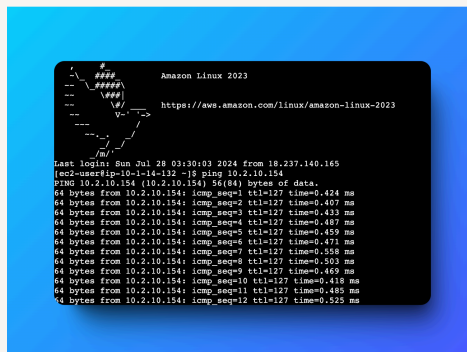


Troubleshooting ping issues

To test VPC peering, I ran the command 'ping 10.2.7.86'

A successful ping test would validate my VPC peering connection because i tried to test the connectivity of my ec2 instance with another ec2 instance in different VPC using its Private IP.

I had to update my second EC2 instance's security group because the default security group of newly created VPC does not allow traffic from outside by default. So i had to explicitly allow ssh inbound of my EC2 Instance in VPC-1 and then allow ICMP inbound of EC2 Instance in VPC-2.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

